

Exemple de soumission d'activité

ÉTS - LOG430 - Architecture logicielle - Été 2026

Étudiant(e) : Chris-Emmanuel Berton

Questions

(Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal pour illustrer chacune de vos réponses.)

1. Question 1 : Quelle réponse obtenons-nous à la requête à POST /payments ? Illustrez votre réponse avec des captures d'écran/du terminal.

Réponse Erreur interne lorsque utilise krakend et problème de permissions lorsque passe par API directement.

```
PS C:\Users\Chris\Documents\ETS\Cours\LOG430-ArchitectureLog\Lab\5\log430-lab5> Invoke-RestMethod `
>> -Uri "http://localhost:8080/payments-api/payments" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{
>>   "user_id": 1,
>>   "order_id": 1,
>>   "total_amount": 100
>> }'
Invoke-RestMethod : Le serveur distant a retourné une erreur : (500) Erreur interne du serveur.
Au caractère Ligne:1 : 1
+ Invoke-RestMethod `
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand
```

```

- Container log430-lab5-redis-1 Restarting
PS C:\Users\Chris\Documents\ETS\Cours\LOG430-ArchitectureLog\Lab\5\log430-lab5> Invoke-RestMethod `
>> -Uri "http://localhost:5009/payments" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{
>>   "user_id": 1,
>>   "order_id": 1,
>>   "total_amount": 100
>> }'
Invoke-RestMethod : {
  "error": "(mysql.connector.errors.ProgrammingError) 1044 (42000): Access denied for user 'labo05'@'%'
to database 'labo05_db'\n(Background on this error at: https://sqlalche.me/e/20/f405)"
}
Au caractère Ligne:1 : 1
+ Invoke-RestMethod `
+ ~~~~~
+ CategoryInfo          : InvalidOperation : (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-RestMethod], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMethodCommand

```

```

payments
payments_api | 172.19.0.1 - - [19/Jun/2026 22:52:30] "POST /payments HTTP
/1.1" 400 -
payments_api | 2026-06-19 22:52:40 - payments - DEBUG - Endpoint: POST /p
payments
payments_api | 172.19.0.9 - - [19/Jun/2026 22:52:40] "POST /payments HTTP
/1.1" 400 -

```

2. Question 2 : Quel type d'information envoyons-nous dans la requête à POST payments/process/:id ? Est-ce que ce serait le même format si on communiquait avec un service SOA, par exemple ? Illustrez votre réponse avec des exemples et captures d'écran/terminal.

```

1  {
2    "cardNumber": 999999999999999,
3    "cardCode": 123,
4    "expirationDate": "2030-01-05"
5  }

```

La requête Post payments/process/:id

envoie les informations de la carte de crédit utilisée pour le paiement Dans le cas d'un service d'une architecture orientée service, le format serait différent puisqu'on utiliserait des interfaces standards tels SOAP et REST, alors que dans cette infrastructure, on utilise le format défini dans le le JSON.

3. Question 3 : Quel résultat obtenons-nous de la requête à POST payments/process/:id?

Les requêtes concernant les orders sont sans problèmes.

The screenshot shows a REST client interface for 'LOG430 Store Manager'. The left sidebar lists collections, with 'LOG430 Store Manager' expanded to show 'Orders'. The main panel displays a POST request to 'LOG430 Store Manager > Orders > /orders'. The request is successful, returning a 201 status code. The response body is a JSON object:

```
{  "order_id": 1}
```

The screenshot shows a REST client interface for 'LOG430 Store Manager'. The left sidebar lists collections, with 'LOG430 Store Manager' expanded to show 'Orders'. The main panel displays a GET request to 'LOG430 Store Manager > Orders > /orders/1'. The request is successful, returning a 201 status code. The response body is a JSON object:

```
{  "items": "[{\"product_id\": 1, \"quantity\": 2}]",  "payment_link": "http://api-gateway:8080/payments-api/payments/process/0",  "total_amount": "3999.98",  "user_id": "1"}
```

Par contre, les requêtes concernant le Payment causent problèmes : dans les deux cas, ils retournent l'erreur 404.

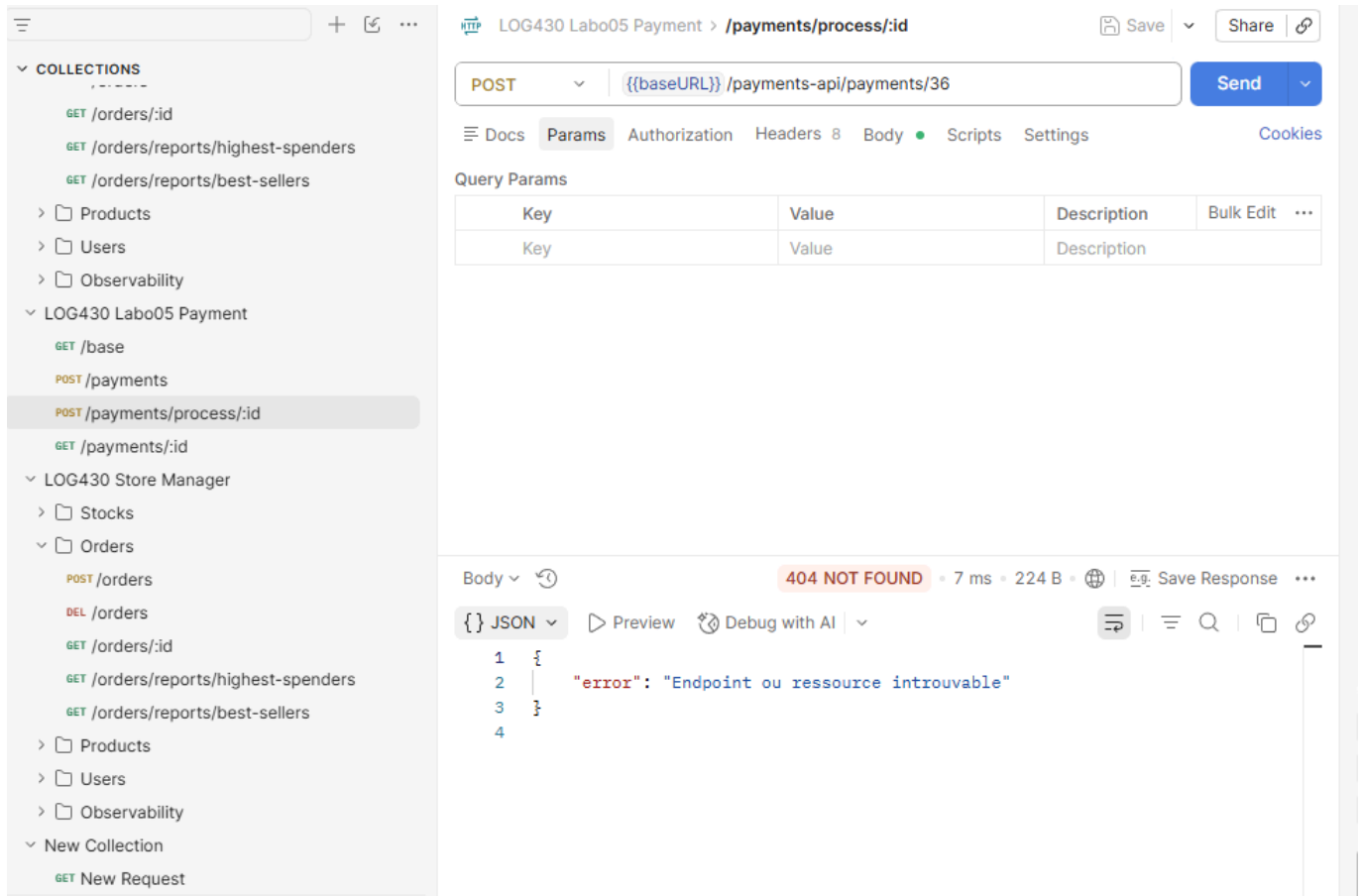
The screenshot shows the Postman interface with a collection named 'LOG430 Labo05 Payment'. The selected request is a POST to `/payments/process/:id`. The URL bar shows `{{baseURL}}/payments-api/payments/36`. The response status is **404 NOT FOUND** with a 7 ms duration and 224 B size. The JSON response body is:

```
{  "error": "Endpoint ou ressource introuvable"}
```

The screenshot shows the Postman interface with the same collection. The selected request is a GET to `/payments/:id`. The URL bar shows `{{baseURL}}/payments/36`. The response status is **404 NOT FOUND** with a 21 ms duration and 365 B size. The JSON response body is:

```
{  "error": "(mysql.connector.errors.ProgrammingError) 1044 (42000):\n    Access denied for user 'labo05@%' to database 'mysql'\n    (Background on this error at: https://sqlalche.me/e/20/f405)"}
```

Le résultat retourne une erreur car il ne reconnaît pas le endpoint.



4. Question 4 : Quelle méthode avez-vous dû modifier dans log430-labo05-payment et qu'avez-vous modifiée ? Justifiez avec un extrait de code.

Réponse

```
def process_payment(payment_id, credit_card_data):
    """ Process payment with given ID, notify store_manager sytem that the order is paid """
    # S'il s'agissait d'un véritable service de paiement, nous utiliserions les données de la carte de crédit pour effectuer le paiement.
    # Cela pourrait se trouver dans un microservice distinct.
    _process_credit_card_payment(credit_card_data)

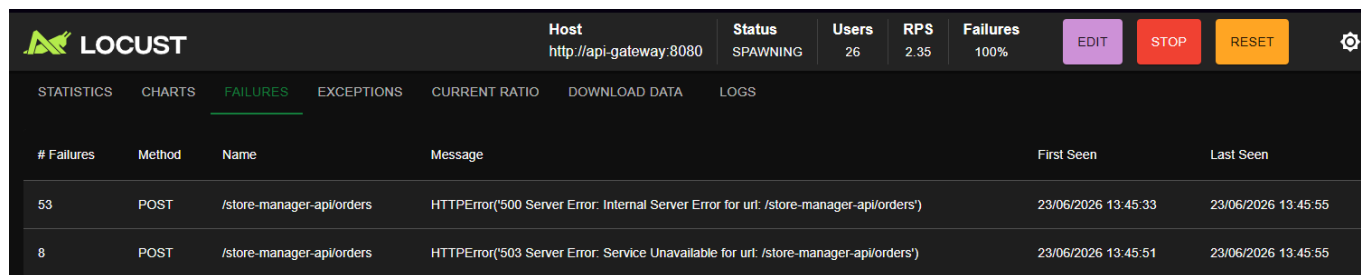
    # Si le paiement est réussi, mettre à jour les statut de la commande
    # Ensuite, faire la mise à jour de la commande dans le Store Manager (en utilisant l'order_id)
    update_result = update_status_to_paid(payment_id)
    logger.debug(f"Updated order {update_result['order_id']} to paid={update_result}")
    result = {
        "order_id": update_result["order_id"],
        "payment_id": update_result["payment_id"],
        "is_paid": update_result["is_paid"]
    }

    requests.put("http://api-gateway:8080/store-manager-api/orders",
        json={ "order_id": update_result["order_id"], "is_paid": True},
        headers={"Content-Type": "application/json"})

    return {
        "order_id": update_result["order_id"],
        "payment_id": update_result["payment_id"],
        "is_paid": True}
```

La méthode modifiée est la méthode `process_payment`. La valeur de retour et la requête ont été modifiés.

5. Question 5 : À partir de combien de requêtes par minute observez-vous les erreurs 503 ? Justifiez avec des captures d'écran de Locust.



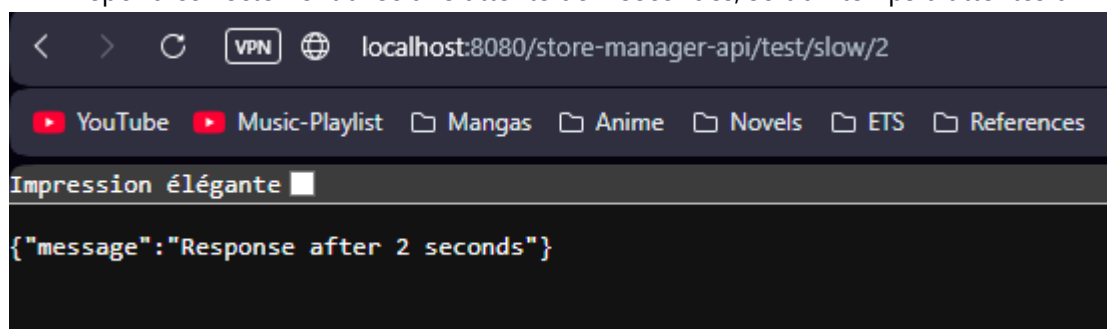
The Locust web interface displays a table of failure statistics. The table has columns for '# Failures', 'Method', 'Name', 'Message', 'First Seen', and 'Last Seen'. Two entries are shown, both for POST requests to '/store-manager-api/orders'.

# Failures	Method	Name	Message	First Seen	Last Seen
53	POST	/store-manager-api/orders	HTTPError('500 Server Error: Internal Server Error for url: /store-manager-api/orders')	23/06/2026 13:45:33	23/06/2026 13:45:55
8	POST	/store-manager-api/orders	HTTPError('503 Server Error: Service Unavailable for url: /store-manager-api/orders')	23/06/2026 13:45:51	23/06/2026 13:45:55

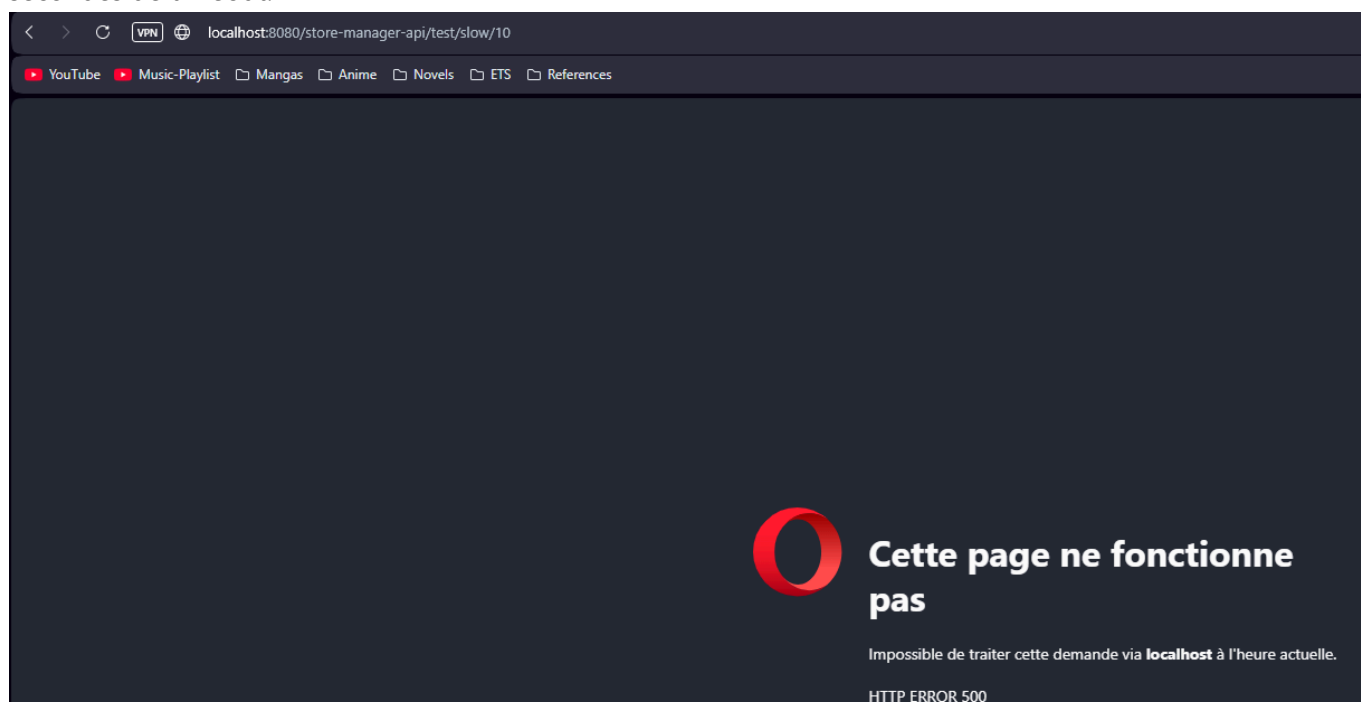
L'erreur 503 apparaît à partir de 2.35 requêtes par secondes, soit 141 requêtes par minutes.

6. Question 6 : Que se passe-t-il dans le navigateur quand vous faites une requête avec un délai supérieur au timeout configuré (5 secondes) ? Quelle est l'importance du timeout dans une architecture de microservices ? Justifiez votre réponse avec des exemples pratiques.

L'API répond correctement avec une attente de 2 secondes, soit un temps d'attente d'inférieure au timeout.



Cependant, le localhost affiche une erreur une fois que le slow atteint 10 secondes, soit au delà des 5 secondes de timeout.



Déploiement

(Le cas échéant, décrivez votre pipeline CI/CD et ce que vous avez appris dans ce laboratoire en ce qui concerne le déploiement. Il est obligatoire d'ajouter du code, des captures d'écran ou des sorties de terminal

pour illustrer votre réponse.)

Ce laboratoire a permis d'utiliser un API Gateway. Dans ce cas-ci, l'API utilisé était KrakenD et à établir une communication inter-services de type Nord-Sud, soit la communication entre le Store-manager et payment (voir) Ce lab nous a aussi permis de valider l'utilité des timeouts (voir Question #6)